Week 4: Advanced SQL

Keeyong Han

Table Of Contents

1. Recap of the Previous Weeks
2. Overview of SQL
3. Basic SQL
4. Advanced SQL
5. Break
6. Methods for Checking Data Quality
7. More on Lab #1
8. Demo & Homework #3
9. Quiz #1

Recap of the Previous Weeks

Key Concepts

- Data Org Evolution
- OLAP vs. OLTP
- Data Pipelines
- Columnar Storage vs. Row Storage
- Primary Key Uniqueness isn’t guaranteed
- Separation of Compute and Storage
- Evolution of Data Infra
- Bulk-update is preferred in Data Warehouse or Data Lake
- Database & schema: a way to organize your tables acting as a hierarchical containers

Data Flow

Site traffic &
3rd party data

1

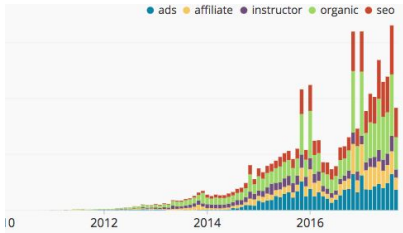
ELT

ETL

Data warehouse

Data Team

2



Business
Insights

3

Site Services

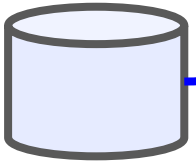
Scalable Product Improvement

OLTP: OnLine Transaction Processing

OLAP: OnLine Analytical Processing

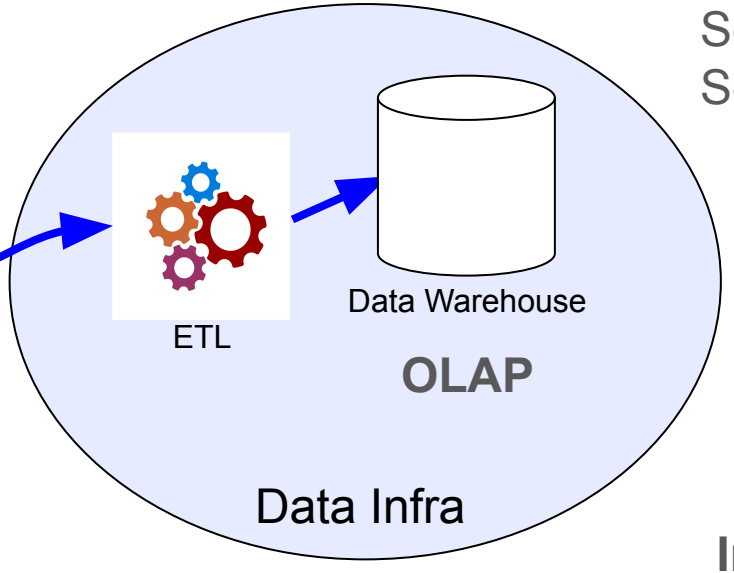
Scale Up

Production DB



OLTP

Scale Up
Scale Out



End Users, Customers

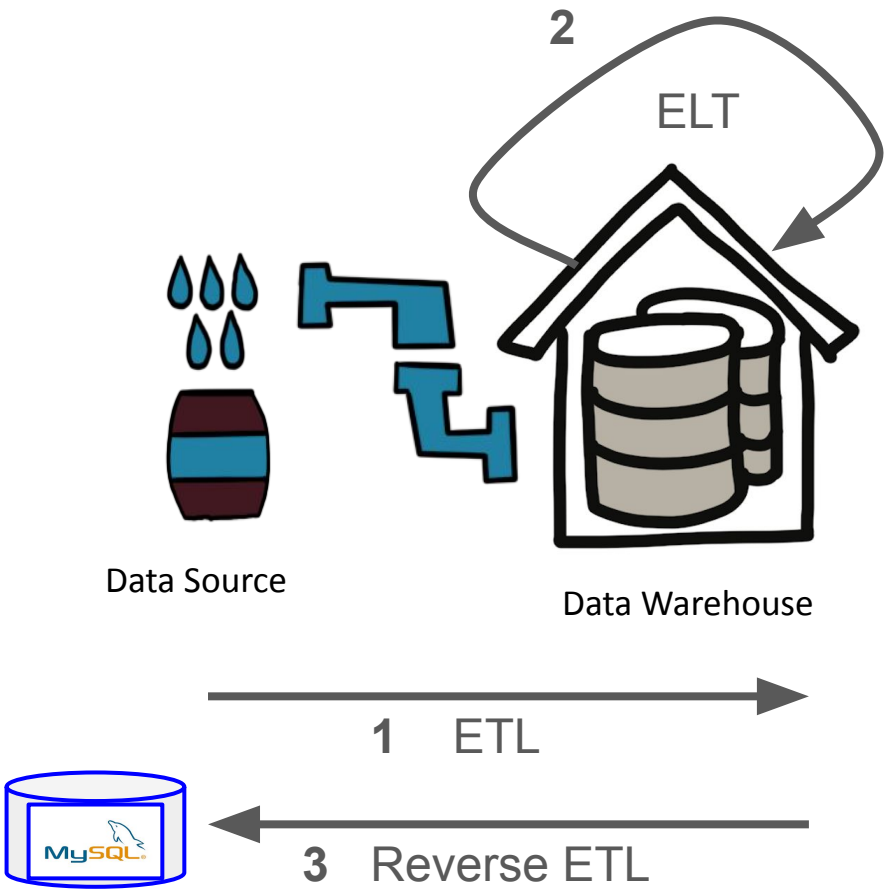
Backend Engineers

Data Engineers

Internal Audience

Types of Data Pipeline

1. ETL
2. ELT (“T”)
3. Reverse ETL



Columnar Storage vs. Row Storage

- Columnar Storage
  - Stores data by column rather than by record
  - Enables column-level compression and very fast addition or deletion of columns

Table

City	revenue
aa	100
bb	200
cc	300

Row Store

Row1	aa 100
Row2	bb 200
Row3	cc 300

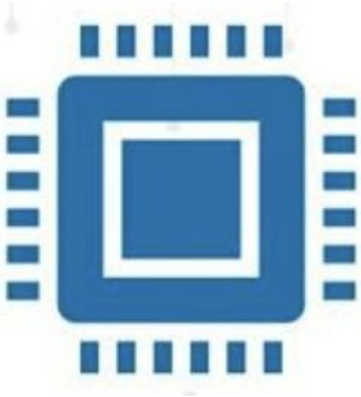
Column Store

City	aa bb cc
revenue	100 200 300

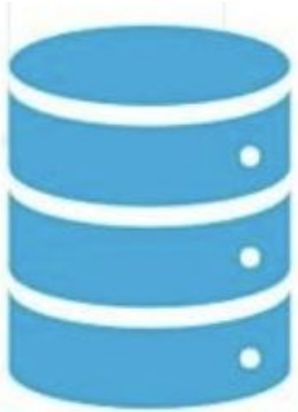
Each is saved as a separate file

Data Warehouse: Compute and Storage

- Storage is about your tables and is cheaper: disk
- Compute is about processing SQL queries and is more expensive: CPU, memory
- Ideally scaling each should be independent



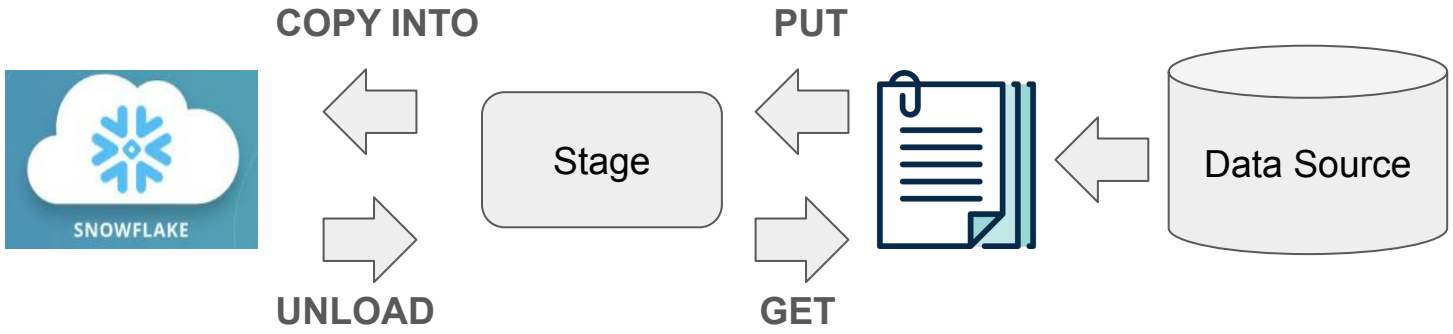
Compute Layer



Storage Layer

Bulk Update in Snowflake (COPY INTO)

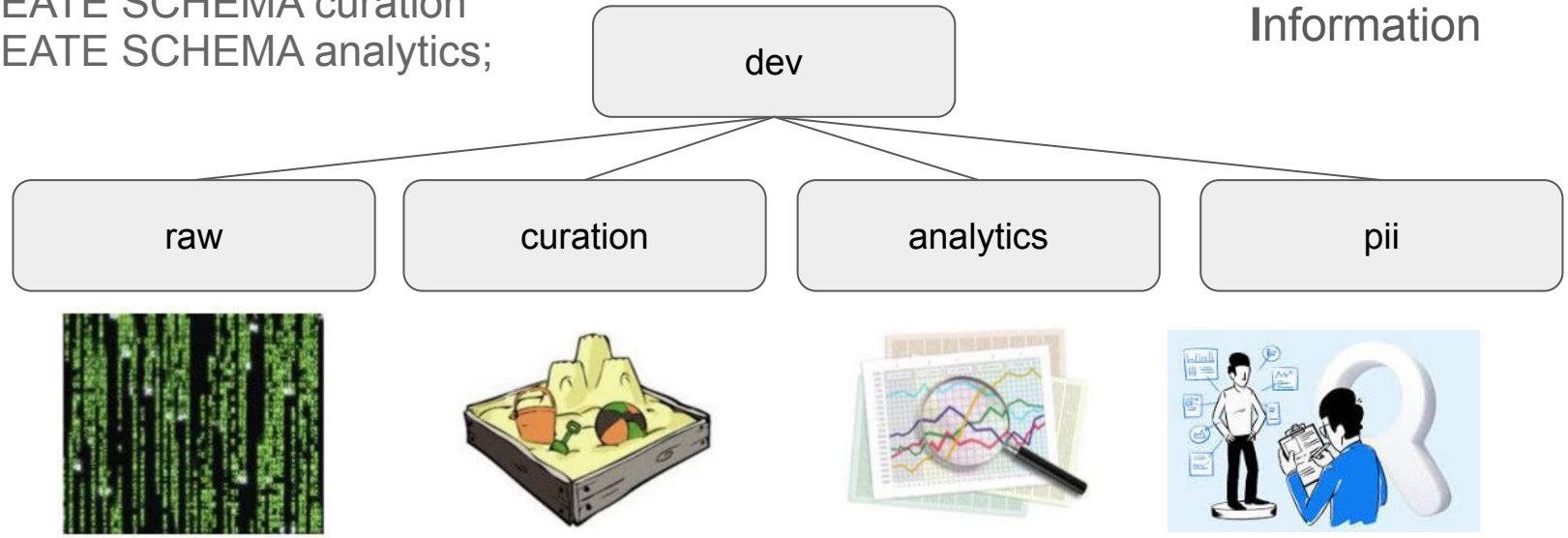
- In Snowflake, the corresponding command is COPY INTO
- This process is done at the file level rather than on a record-by-record basis, using an intermediary called a “stage”
- Conversely, you can also download data from Snowflake to a stage using the UNLOAD command.



Database ➔ Schema: a way to organize your tables

```
CREATE DATABASE dev;
CREATE SCHEMA raw;
CREATE SCHEMA curation
CREATE SCHEMA analytics;
```

Personally
Identifiable
Information



ETL Tables

Cleanup

ELT Tables

PII Tables

Snowflake Characteristics (1)

- Storage layer and Compute layer are separated
- Compute layer is called “Virtual WareHouse”
  - Two types of Virtual WareHouse exist: Regular & Snowpark
  - A Virtual WH has a size: from X-Small to 6X-Large
- Storage layer supports “Time Travel” and “Zero Copy Cloning”
- Streaming processing is supported
- Runs on top of AWS, GCP and Azure
- For bulk-update, stage is used as a middle ground: internal vs. external
  - COPY INTO

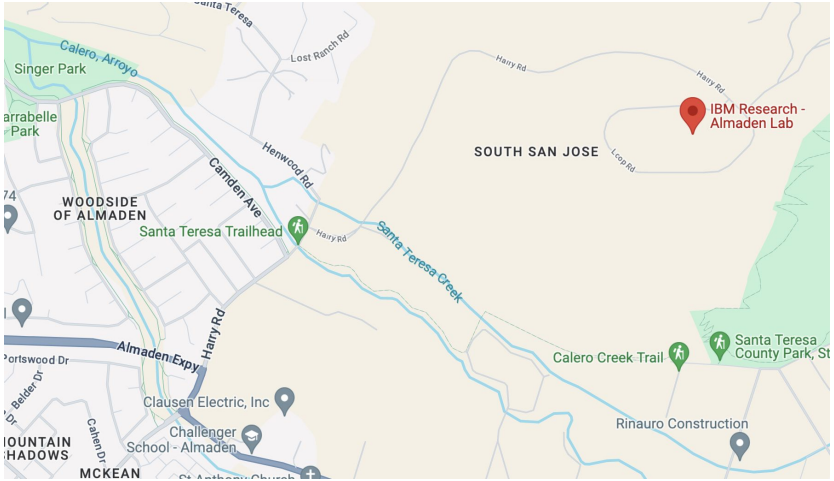
Snowflake Characteristics (2)

- Account Structure: Organization -> 1+ Account -> 1+ Databases
- Data Marketplace & Data Sharing (“Share, Don’t Move”)
- Snowflake has 4 options
  - Standard, Enterprise, Business Critical and Virtual Private Snowflake
- Pricing itself has 3 components
  - Compute Costs: Determined by credits. A credit costs
    - Standard: $2, Enterprise: $3, Business-critical: $4
    - Virtual Warehouse size will determine credit consumption
      - X-Small: 1 credit / hour, …, 6X-Large: 512 credits / hour
  - Storage Costs: Calculated per terabyte (TB)
  - Network Costs: Calculated per TB for data transfers

Overview of SQL

History of SQL

- SQL: Structured Query Language
- SQL was developed by IBM Almaden Center in early 1970s
- A language to manipulate tables in Relational Databases
  - Best in handling structured data
  - Survived in the era of Big Data



Components of SQL

- DDL (Data Definition Language):
  - SQL language for defining table structure (create,drop,alter)
- DML (Data Manipulation Language):
  - Query language for retrieving desired records from a table
    - SELECT
  - Language used for adding/deleting/updating records in a table
    - INSERT, DELETE, UPDATE, COPY, …

Advantage of SQL

- SQL is used for structured data regardless of data scale
- All large-scale data warehouses are SQL-based
  - Redshift, Snowflake, BigQuery, ClickHouse, …
- Spark and Hadoop are no exception
  - SQL languages like SparkSQL and HiveQL are supported
- A fundamental skill in the data field
  - Data engineers, data analysts, and data scientists all need to know it
  - Anyone who needs to work with data should know it, regardless of their job roles

Disadvantage of SQL

- SQL alone cannot process unstructured data
  - Can handle unstructured data to some extent using regular expressions with limitations
  - Many relational databases only support flat structures (no nesting like JSON)
    - Google BigQuery and Snowflake support **nested** structures though
  - Spark/Pandas (“dataframes”) are needed to handle unstructured data
- SQL syntax varies slightly between different relational databases

```
{
 "isbn": "123-456-222",
 "author":
   {
     "lastname": "Doe",
     "firstname": "Jane"
   },
 "title": "The Ultimate Database Study Guide",
 "category": ["Non-Fiction", "Technology"]
}
```

An Example of Converting semi structured data to structured

47.29.201.179 - - [28/Feb/2019:13:17:10 +0000] "GET /?p=1 HTTP/2.0" 200 5316 "https://domain1.com/?p=1" "Mozilla/5.0 (Windows NT 6.1) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/72.0.3626.119 Safari/537.36" "2.75"

REGEX

remote_ip,
remote_user,
time,
method,
uri,
protocol,
status_code, …

Basic SQL

DDL, DML, WHERE, GROUP BY, COUNT, DISTINCT

DDL

- CREATE TABLE
  - CREATE TABLE … AS SELECT: CTAS
- DROP TABLE
- ALTER TABLE

```
CREATE TABLE IF NOT EXISTS dev.raw.user_session_channel (
    userId int not NULL,
    sessionId varchar(32) primary key,
    channel varchar(32) default 'direct'
);
```

```
DROP TABLE IF EXISTS dev.raw_data.user_session_channel;
```

```
ALTER TABLE dev.raw_data.user_session_channel RENAME COLUMN channel to channelName;
```

DML Overview

- Record loading
  - INSERT INTO
  - COPY
- Record deletion: DELETE
- Record update: UPDATE
- Record query: SELECT
  - GROUP BY
  - JOIN
  - UNION, INTERSECT, …
  - WINDOW function

SELECT

- A query language used to utilize information stored in tables.
- It also extracts new information from the values of fields.
  - Transforms using functions, arithmetic operations, CASE WHEN, etc.
- Groups records with the same field values and performs various operations
  - GROUP BY
  - Performs aggregations like count, sum, average, standard deviation, etc.
- Combines multiple tables to generate new information.
  - JOIN
  - UNION

SELECT Syntax

- Used to read records (or the number of records) from tables.
- Uses WHERE to select records that meet specific conditions.

```
SELECT [DISTINCT] field1, field2, …
FROM table1
[JOIN table2 ON JOIN condition]
[WHERE filter_condition]
[GROUP BY field1, field2, …]
[ORDER BY field1 [ASC|DESC]]
[LIMIT N];
```

fields can be transformed (CASE WHEN, functions, arithmetic operations, …)

COUNT Function

value
NULL
1
1
0
0
4
3

Table: adhoc.count_test

SELECT COUNT(1) FROM adhoc.count_test	7
SELECT COUNT(0) FROM adhoc.count_test	7
SELECT COUNT(NULL) FROM adhoc.count_test	0
SELECT COUNT(value) FROM adhoc.count_test	6
SELECT COUNT(DISTINCT value) FROM adhoc.count_test	4

CASE WHEN

value	sign
NULL	null
1	positive
1	positive
0	zero
0	zero
4	positive
3	positive

Table: adhoc.count_test

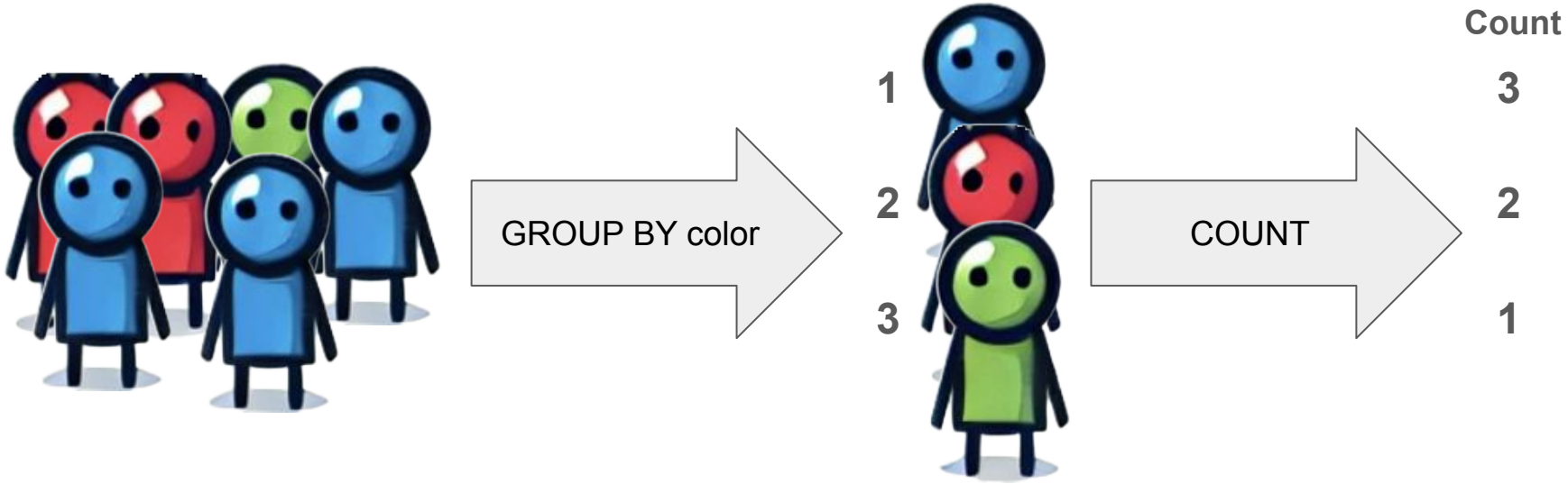
```
SELECT
    value,
    CASE
        WHEN value > 0 THEN 'positive'
        WHEN value = 0 THEN 'zero'
        WHEN value < 0 THEN 'negative'
        ELSE 'null'
    END sign
FROM dev.adhoc.count_test;
```

What is NULL?

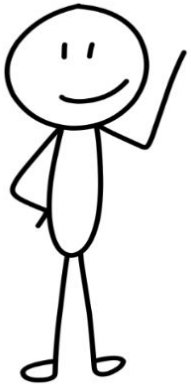
- A constant that indicates the absence of a value.
  - It is different from 0 or "" (empty string), which represent some values
- When specifying a field, NULL or NOT NULL can be set
  - Primary key field can’t have NULL
- Comparing whether a field's value is NULL requires special syntax:
  - Field1 IS NULL or field1 IS NOT NULL
  - ~~field1 = NULL or field1 != NULL~~
- If NULL is used in arithmetic operations, what will be the result?
  - SELECT 0 + NULL, 0 - NULL, 0 * NULL, 0/NULL

Common Data Analysis Cases

- Create groups of records based on specific field(s) and compare them.
  a. The GROUP BY clause is used for this purpose: it creates groups based on specific field(s).
- Within each group, aggregate functions are used to calculate specific statistics.



Web Service: User ID & Session ID



User 100

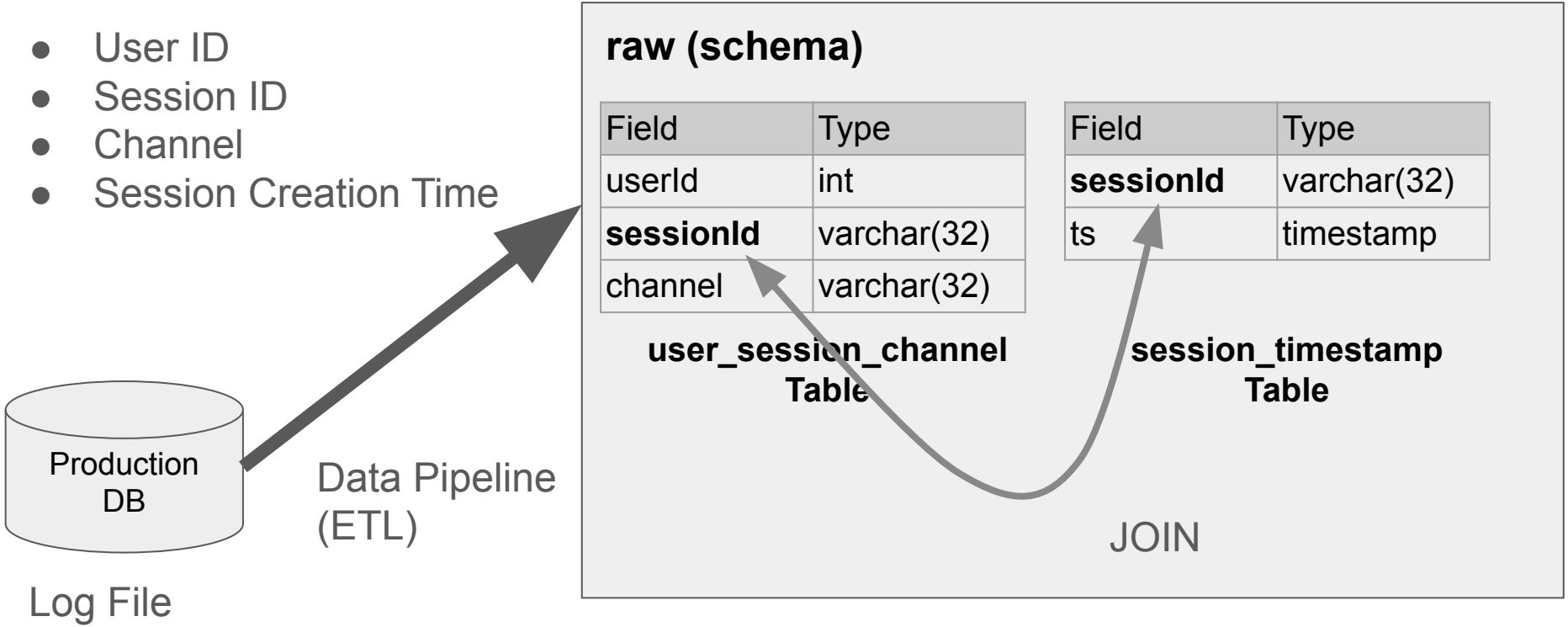
Every page visits from User ID 100

First Visit via Google Search	Revisit 30 minutes later	Revisit via Instagram
Session #1	Session #2	Session #3

UserID	SessionID	Channel	Session Creation Time

Tables to use in some practices

- User ID
- Session ID
- Channel
- Session Creation Time



GROUP BY & Aggregate Examples

```
SELECT channel, COUNT(1) AS cnt
FROM dev.raw.user_session_channel
GROUP BY channel      -- it can be 1 instead
ORDER BY cnt DESC;  -- it can be 2 instead
```

GROUP BY & Aggregate Functions

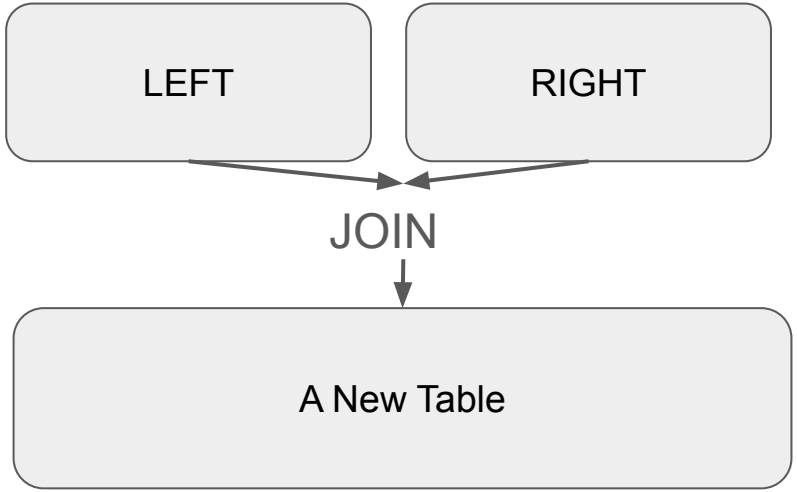
- Group the records of a table and calculate various information for each group.
- This process consists of two steps:
  - First, decide the field(s) to group by (can be one or more fields).
    - Specify these fields using GROUP BY (using field names or field ordinal numbers).
  - Next, decide what to calculate for each group.
    - Here, aggregate functions are used, such as **COUNT, SUM, AVG, MIN, MAX**, etc.
    - It is common to specify field names (using aliases).
      - For example: COUNT(1) AS cnt

SQL Practice #1

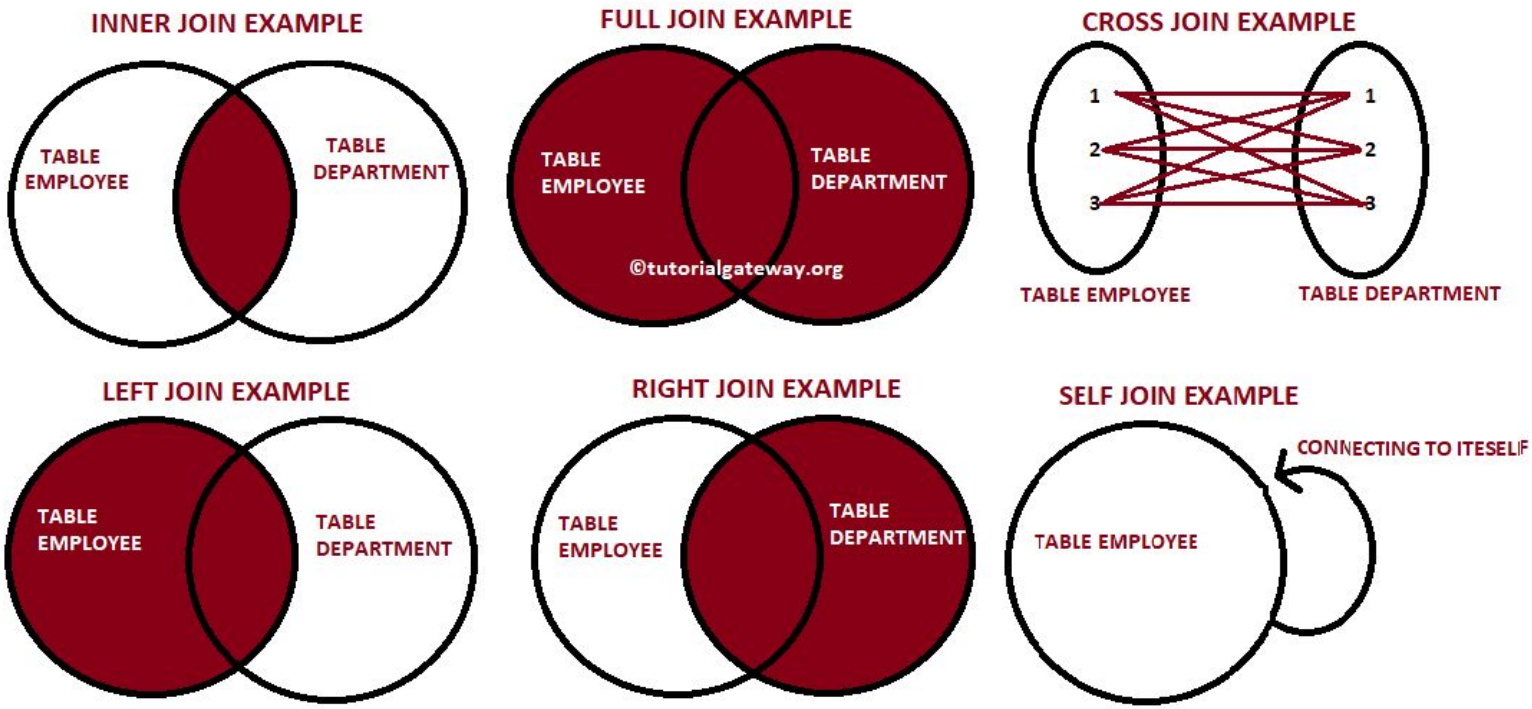
- Course GitHub Repo: https://github.com/keeyong/sjsu-data226-SP26/
- DDL
- DML: INSERT, UPDATE, DELETE
- SELECT
  - CASE WHEN
  - COUNT
  - NULL
  - GROUP BY

What is JOIN?

- JOIN merges two or more tables using a common field ("join key")
  - Used to integrate information that was dispersed across multiple tables
- JOIN creates a new table containing fields from both sides
- Depending on the join method, two things differ:
  - Which records are selected?
  - How are the fields populated?



Types of JOIN



Source: https://theartofpostgresql.com/blog/2019-09-sql-joins/

JOIN Syntax

```
SELECT A.*, B.*
FROM raw_data.table1 A
____ JOIN raw.table2 B ON A.key1 = B.key1 and A.key2 = B.key2
WHERE A.ts >= '2019-01-01';
```

INNER, FULL, LEFT, RIGHT, CROSS

Two Tables for JOIN practices

UserID	VitalID	Date	Weight
100	1	2020-01-01	75
100	3	2020-01-02	78
101	2	2020-01-01	90
101	4	2020-01-02	95

raw.vital

AlertID	VitalID	AlertType	Date	UserID
1	4	WeightIncrease	2020-01-02	101
2	NULL	MissingVital	2020-01-04	100
3	NULL	MissingVital	2020-01-04	101

raw.alert

JOIN KEY

INNER JOIN

1. Returns only the records that match from both tables
2. Returns with all fields from both tables populated

```
SELECT * FROM raw_data.Vital v
JOIN raw_data.Alert a ON v.vitalID = a.vitalID;
```

v.UserID	v.VitalID	v.Date	v.Weight	a.AlertID	a.VitalID	a.AlertType	a.Date	a.UserID
101	4	2020-01-02	95	1	4	WeightIncrease	2021-01-02	101

LEFT JOIN

1. Returns all records from the left table (Base)
2. Fields from the right table are populated only when they match a record from the left table

```
SELECT * FROM raw_data.Vital v
LEFT JOIN raw_data.Alert a ON v.vitalID = a.vitalID;
```

v.UserID	v.VitalID	v.Date	v.Weight	a.AlertID	a.VitalID	a.AlertType	a.Date	a.UserID
100	1	2020-01-01	75	NULL	NULL	NULL	NULL	NULL
100	3	2020-01-02	78	NULL	NULL	NULL	NULL	NULL
101	2	2020-01-01	90	NULL	NULL	NULL	NULL	NULL
101	4	2020-01-02	95	1	4	WeightIncrease	2021-01-02	101

FULL JOIN

1. Returns all records from both the left table and the right table
2. All fields from both tables are populated only when there's a match

```
SELECT * FROM raw_data.Vital v
FULL JOIN raw_data.Alert a ON v.vitalID = a.vitalID;
```

v.UserID	v.VitalID	v.Date	v.Weight	a.AlertID	a.VitalID	a.AlertType	a.Date	a.UserID
100	1	2020-01-01	75	NULL	NULL	NULL	NULL	NULL
100	3	2020-01-02	78	NULL	NULL	NULL	NULL	NULL
101	2	2020-01-01	90	NULL	NULL	NULL	NULL	NULL
101	4	2020-01-02	95	1	4	WeightIncrease	2021-01-02	101
NULL	NULL	NULL	NULL	2	NULL	MissingVital	2020-01-04	100
NULL	NULL	NULL	NULL	3	NULL	MissingVital	2020-01-04	101

CROSS JOIN

1. Returns all combinations of records from the left table and the right table

```
SELECT *
FROM raw_data.Vital v
CROSS JOIN raw_data.Alert a;
```

v.UserID	v.VitalID	v.Date	v.Weight	a.AlertID	a.VitalID	a.AlertType	a.Date	a.UserID
100	1	2020-01-01	75	1	4	WeightIncrease	2020-01-01	101
100	3	2020-01-02	78	1	4	WeightIncrease	2020-01-01	101
101	2	2020-01-01	90	1	4	WeightIncrease	2020-01-01	101
101	4	2020-01-02	95	1	4	WeightIncrease	2020-01-01	101
100	1	2020-01-01	75	2		MissingVital	2020-01-04	100
100	3	2020-01-02	78	2		MissingVital	2020-01-04	100
101	2	2020-01-01	90	2		MissingVital	2020-01-04	100
101	4	2020-01-02	95	2		MissingVital	2020-01-04	100
100	1	2020-01-01	75	3		MissingVital	2020-01-04	101
100	3	2020-01-02	78	3		MissingVital	2020-01-04	101
101	2	2020-01-01	90	3		MissingVital	2020-01-04	101
101	4	2020-01-02	95	3		MissingVital	2020-01-04	101

SELF JOIN

1. Joins a table with itself using different aliases

```
SELECT *
FROM raw_data.Vital v1
JOIN raw_data.Vital v2 ON v1.vitalID = v2.vitalID;
```

v1.UserID	v1.VitalID	v1.Date	v1.Weight	v2.UserID	v2.VitalID	v2.Date	v2.Weight
100	1	2020-01-01	75	100	1	2020-01-01	75
100	3	2020-01-02	78	100	3	2020-01-02	78
101	2	2020-01-01	90	101	2	2020-01-01	90
101	4	2020-01-02	95	101	4	2020-01-02	95

SQL Practice #2

- INNER JOIN
- LEFT JOIN (RIGHT JOIN)
- FULL JOIN
- CROSS JOIN
- SELF JOIN

Advanced SQL

CTAS, UNION/INTERSECT/EXCEPT, Window Function

CTAS: The Simplest ELT

- A simple way to create a new table from existing tables
- If you have to join tables frequently,
  - Periodically create the table with the same JOINs using CTAS
  - Tools like Airflow are often used for this scheduling

```
CREATE TABLE analytics.session_summary AS
SELECT B.*, A.ts
FROM raw.session_timestamp A
JOIN raw.user_session_channel B ON A.sessionid = B.sessionid;
```

MAU calculation with the ELT table (analytics.session_summary)

```
SELECT
  LEFT(ts, 7) AS year_month,
  COUNT(DISTINCT userid) AS mau
FROM analytics.session_summary
GROUP BY 1
ORDER BY 1 DESC;
```

MAU: Monthly Active User
DAU: Daily Active User
WAU: Weekly Active User

CTE (Common Table Expression)

- Create and use temporary tables before performing a SELECT as a part of SELECT
  - This temp tables are created as a part of SELECT and will disappear later
  - They are created in a single SQL statement at the beginning of the SELECT query
- The syntax is as follows (creating temp tables named 'channel' and 'temp')

```
WITH channel AS (
   select DISTINCT  channel from raw_data.user_session_channel
),
temp AS (
   select …
),
…
SELECT *
FROM channel c
JOIN temp t ON c.userId = t.userId
```

CTE based MAU computation

```
WITH tmp AS (
    SELECT B.*, A.ts
    FROM raw_data.session_timestamp A
    JOIN raw_data.user_session_channel B ON A.sessionid = B.sessionid
)
SELECT
   LEFT(ts, 7) AS year_month,
   COUNT(DISTINCT userid) AS mau
FROM tmp
GROUP BY 1
ORDER BY 1 DESC;
```

Subquery based MAU computation

```
SELECT
   LEFT(ts, 7) AS year_month,
   COUNT(DISTINCT userid) AS mau
FROM (
    SELECT B.*, A.ts
    FROM raw_data.session_timestamp A
    JOIN raw_data.user_session_channel B ON A.sessionid = B.sessionid
)
GROUP BY 1
ORDER BY 1 DESC;
```

SQL Practice #3

- CTAS
- CTE & Subquery

Break

NULLIF

- NULLIF(num1, num2): if num1=num2, then return NULL else return num1
- num1/num2
  - if num2 is 0, then it causes a "divide by 0" error
- How to prevent this? Use NULLIF to change 0 to NULL
  - num1/NULLIF(num2, 0)
  - Remember once again: if NULL is used in arithmetic operations, the result becomes NULL!

COALESCE

- A function that replaces NULL values with other values,
- COALESCE(exp1, exp2, exp3, …)
  - The function checks each argument starting from exp1, and returns the first non-NULL value it encounters.
  - If all arguments are NULL, it will ultimately return NULL.

```
SELECT
    value,
    COALESCE(value, 0)  -- If value is NULL, return 0
FROM raw_data.count_test;
```

value
NULL
1
1
0
0
4
3

raw_data.count_test

UNION, EXCEPT, INTERSECT

- Each SELECT statement must have matching # of fields and types
- UNION (Union)
  - Combines multiple tables or SELECT results into a single result.
  - UNION vs. UNION ALL
    - UNION removes duplicates.
- EXCEPT (Difference)
  - Allows you to subtract one SELECT result from another.
- INTERSECT (Intersection)
  - Finds only the common records among multiple SELECT statements.

Examples of UNION, EXCEPT, and INTERSECT

```
SELECT 'mark' as first_name, 'zuckerberg' as last_name
UNION  -- UNION ALL
SELECT 'elon', 'musk'
UNION
SELECT 'mark', 'zuckerberg';

SELECT sessionId FROM raw_data.user_session_channel
EXCEPT
SELECT sessionId FROM raw_data.session_transaction;

SELECT sessionId FROM raw_data.user_session_channel
INTERSECT
SELECT sessionId FROM raw_data.session_transaction;
```

WINDOW Functions

- Unlike GROUP BY aggregation functions, Window functions don’t collapse rows. In other words, the number of records doesn’t change
- Syntax:
  - Window_functions OVER (PARTITION BY … ORDER BY …)
- Will learn a few functions
  - **ROW_NUMBER**
  - **LAG**
  - FIRST_VALUE/LAST_VALUE
  - SUM

WINDOW Functions - ROW_NUMBER (1)

userid	ts	channel
10	2021-01-01	google
11	2021-01-03	facebook
11	2021-01-01	naver
10	2021-01-02	facebook
11	2021-01-04	google
10	2021-01-03	youtube

userid	ts	channel	nn
10	2021-01-01	google	1
10	2021-01-02	facebook	2
10	2021-01-03	youtube	3
11	2021-01-01	naver	1
11	2021-01-03	facebook	2
11	2021-01-04	google	3

3. ROW_NUMBER can be used to implement

1. What if you want to assign a sequential number per user based on the timestamp?

2. Add a new column!
Partition records by user, sort them by time within each group, and assign numbers starting from 1.

WINDOW Functions - ROW_NUMBER (2)

- Assigns a unique sequential number to each row within a result set defined by **ORDER BY** and **PARTITION BY**
- What if you want to assign a sequential number per user based on the timestamp?

```
SELECT usc.userid, usc.channel, ROW_NUMBER() OVER (PARTITION BY usc.userid ORDER BY st.ts) nn
FROM raw_data.user_session_channel usc
JOIN raw_data.session_timestamp st ON usc.sessionid = st.sessionid;
```

WINDOW Functions - LAG Function (1)

- When all sessions are sorted in ascending order by time for each user
  - What is the channel of the next session?
  - What is the channel of the previous session?

userId	sessionId	channel	ts	previous channel
27	a67c8c9a961b4182688768dd9ba015fe	Youtube	2019-05-01 17:04:00	
27	b04c387c8384ca083a71b8da516f65f6	Google	2019-05-02 19:21:30	Youtube
27	abebb7c39f4b5e46bbcfab2b565ef32b	Naver	2019-05-03 20:38:41	Google
27	ab49ef78e2877bfd2c2bfa738e459bf0	Facebook	2019-05-04 21:48:07	Naver
27	f740c8d9c193f16d8a07d3a8a751d13f	Facebook	2019-05-05 18:15:31	Facebook

```
LAG(channel,1) OVER (PARTITION BY userId ORDER BY ts) prev_channel
```

WINDOW Functions - LAG Function (2)

```
-- Find the previous channel
SELECT usc.*, st.ts,
    LAG(channel, 1) OVER (PARTITION BY userId ORDER BY ts) prev_channel
FROM raw_data.user_session_channel usc
JOIN raw_data.session_timestamp st ON usc.sessionid = st.sessionid
ORDER BY usc.userid, st.ts
```

userId	sessionId	channel	ts	previous channel
27	a67c8c9a961b4182688768dd9ba015fe	Youtube	2019-05-01 17:04:00	
27	b04c387c8384ca083a71b8da516f65f6	Google	2019-05-02 19:21:30	Youtube
27	abebb7c39f4b5e46bbcfab2b565ef32b	Naver	2019-05-03 20:38:41	Google
27	ab49ef78e2877bfd2c2bfa738e459bf0	Facebook	2019-05-04 21:48:07	Naver
27	f740c8d9c193f16d8a07d3a8a751d13f	Facebook	2019-05-05 18:15:31	Facebook

SQL Practice #4

- NULLIF, COALESCE
- UNION, EXCEPT, INTERSECT
- WINDOW FUNCTIONS
  - ROW_NUMBER, LAG, FIRST_VALUE/LAST_VALUE, …

Methods for Checking Data Quality

Data quality checks that should always be attempted

- Check for Duplicate Records
- Check for the Presence of Recent Data (Freshness)
- Check if Primary Key Uniqueness is Maintained
- Check for Columns with Missing Values

- When using CTAS, it is important to apply the following tests:
  - Input Tables
  - Output Tables

```
CREATE TABLE analytics.session_summary AS
SELECT B.*, A.ts
FROM raw_data.session_timestamp A
JOIN raw_data.user_session_channel B ON A.sessionid = B.sessionid;
```

Checking for duplicate records

- Compare the following two counts:

```
SELECT COUNT(1)
FROM analytics.session_summary;

SELECT COUNT(1)
FROM (
    SELECT DISTINCT *
    FROM analytics.session_summary
);
```

Checking for the Presence of Recent Data (Freshness)

- Find timestamp fields, check the range (min & max) and see if it is within your expectation

```
SELECT MIN(ts), MAX(ts)
FROM analytics.session_summary;
```

Checking for primary key uniqueness

- Group by the primary key and count. See if any count is bigger than 1

```
SELECT sessionId, COUNT(1)
FROM analytics.session_summary
GROUP BY 1
ORDER BY 2 DESC
LIMIT 1;
```

Check for columns with missing values

```
SELECT
  COUNT(CASE WHEN sessionId is NULL THEN 1 END) sessionid_null_count,
  COUNT(CASE WHEN userId is NULL THEN 1 END) userid_null_count,
  COUNT(CASE WHEN ts is NULL THEN 1 END) ts_null_count,
  COUNT(CASE WHEN channel is NULL THEN 1 END) channel_null_count
FROM analytics.session_summary;
```

dbt to the Rescue

- Data Build Tool (https://www.getdbt.com/)
  - ELT Open source by dbt Labs ($4.2B valuation as of 2022)
    - Provides a Cloud version (dbt Cloud)
  - Coined a term called “Analytics Engineer”
- Supports various Data Warehouses
  - Redshift, Snowflake, Bigquery, Spark, DuckDB, …
- Competitors
  - Coalesce
  - SQLMesh

What you can do with dbt

- Data quality testing and error reporting
- Ability to check data lineage between datasets
- Incremental update of fact tables
- Change tracking for dimension tables (history tables)
- Easy document creation
- Git integration for version control and collaboration

SQL Practice #5

- Table Data Quality Validations

More on Lab #1

Lab 1: Building a Weather Data Analytics

- Make sure you talked to your partner
- Place source codes (Python and SQL) in GitHub
- ETL & Weather prediction are the main topics
- Grading details will be provided in the Files -> Lab

Report Format

- Team Introduction
- Problem Statement
- Dataset(s)
- System Diagram
- Tables
- Implementation Details
- Lessons
- Future Work
- Conclusion

Use screenshots as many as possible
Submit the GitHub link of the project
- GitHub repos should have README.md

Demo: Advanced Snowflake Features

Forecasting

DEMO (Forecasting in Snowflake)

- https://github.com/keeyong/sjsu-data226-SP26/blob/main/week4/ML-Forecasting-Against-Stock-Data.sql

What is ML Forecasting?

- Learn patterns from historical time series data using machine learning
- Predict future events or values
- Examples: stock prices, sales amounts, demand fluctuations, etc.

Homework #3

Weather Temperature Forecasting (13 pt)

SanJose.csv file is under Files/Homeworks/ on Canvas
Please follow the demo as a reference and complete the below assignment

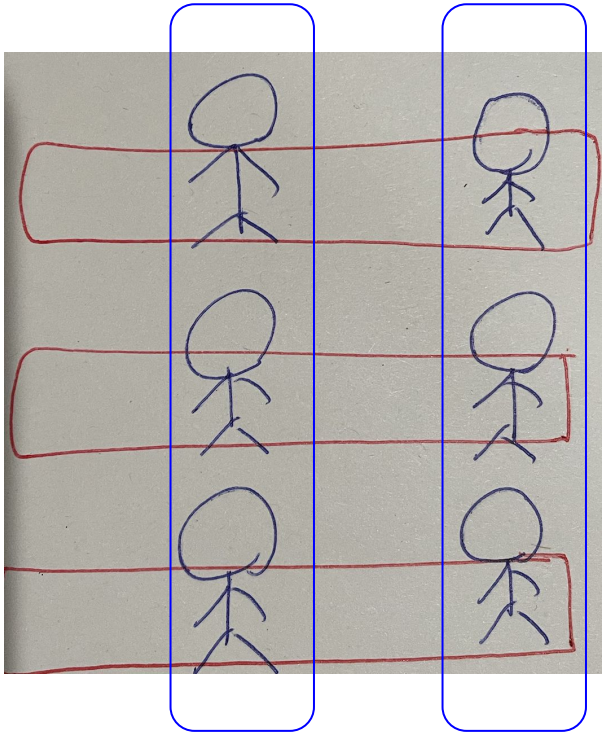
1. (+2) Create a Table Weather_DATA under raw schema with SanJose.csv
2. (+3) Create a view to forecast city of 'SanJose'
   - temp_max should be the forecasting target
3. (+3) Create a forecast model ‘weather_temperature’
4. (+1) Display the Results (historical + predicted together)
   - Run the forecast for the next 7days, and display the results together with the historical data in a single query output by using UNION
5. (+3) Explain your understanding about the Forecasting Process.

(+1) For following the format

Quiz #1

Ground Rules

- 15 questions in 15 minutes
- No bathroom breaks allowed
- Put your digital devices in your own bags.
- Keep your bags on the floor
- Seating Criteria
  - Aligned in the same vertical columns.
  - Seats must be alternated within each row, with no students sitting directly beside each other.



Next Week: Data Pipeline Overview

Refresh Your Python Knowledge

Behavioral Interview Question